Lekcja 5. Preprocesor i dyrektywy preprocesora

Przedmiot: Programowanie aplikacji | Zawód: technik programista | Klasa 4 | Kwalifikacja: INF.04

1. Cele lekcji

- wyjaśnisz, czym jest preprocesor i w którym momencie działa w procesie translacji programu C++;
- zastosujesz dyrektywy #include, #define, #undef oraz dyrektywy kompilacji warunkowej;
- odróżnisz pliki nagłówkowe biblioteki standardowej od plików nagłówkowych tworzonych przez programistę;
- utworzysz prosty projekt wieloplikowy z plikiem nagłówkowym i źródłowym;
- ocenisz, kiedy makro jest użyteczne, a kiedy lepiej użyć const, constexpr, using albo funkcji inline;
- przeanalizujesz kod z makrodefinicjami i wskażesz możliwe błędy.

Powiązanie z INF.04: temat wspiera projektowanie, programowanie i testowanie aplikacji, w szczególności organizowanie kodu programu, stosowanie bibliotek, przygotowanie wieloplikowych projektów oraz analizę działania programu przed kompilacją.

2. Nowe skróty i pojęcia

Element	Znaczenie
C++	Język programowania używany do tworzenia aplikacji; nazwa czytana często jako „C plus plus”.
preprocesor	Etap przetwarzania kodu wykonywany przed właściwą kompilacją. W C++ odpowiada m.in. za dołączanie plików, podstawianie makr i kompilację warunkową.
dyrektywa	Polecenie dla preprocesora. Rozpoczyna się znakiem #, np. #include, #define, #ifdef.
makro	Zamiennik tekstowy rozwijany przez preprocesor. Może reprezentować stałą, fragment kodu albo konstrukcję podobną do funkcji.
IDE	Integrated Development Environment - zintegrowane środowisko programistyczne, np. Visual Studio, Code::Blocks, CLion.
API	Application Programming Interface - zestaw funkcji, klas lub reguł umożliwiający korzystanie z danego narzędzia albo biblioteki.
header guard	Zabezpieczenie pliku nagłówkowego przed wielokrotnym dołączeniem, najczęściej z użyciem #ifndef, #define, #endif.

3. Gdzie działa preprocesor?

Zanim kompilator zacznie tłumaczyć kod źródłowy C++ na kod wynikowy, kod jest przetwarzany przez preprocesor. Preprocesor nie wykonuje programu. On przygotowuje tekst programu dla kompilatora: dołącza wskazane pliki, podstawia zdefiniowane zamienniki, usuwa lub pozostawia fragmenty kodu zależnie od warunków.

SCHEMAT: Kod źródłowy .cpp -> preprocesor -> kod po rozwinięciu dyrektyw -> kompilator -> plik obiektowy -> linker -> program wykonywalny.

W praktyce oznacza to, że błąd w dyrektywie preprocesora może uniemożliwić kompilację jeszcze zanim kompilator sprawdzi typy danych, zgodność parametrów funkcji albo poprawność instrukcji języka C++.

4. Dyrektywa #include

Dyrektywa #include służy do dołączania zawartości innego pliku do aktualnego pliku źródłowego. Dzięki niej można korzystać z bibliotek standardowych oraz z własnych plików nagłówkowych.

Element	Znaczenie
#include <iostream>	Dołączenie pliku nagłówkowego z katalogów biblioteki standardowej lub katalogów skonfigurowanych w środowisku.
#include "operacje.h"	Dołączenie pliku nagłówkowego użytkownika. Preprocesor zwykle najpierw szuka pliku w katalogu bieżącego projektu.

Przykład użycia bibliotek standardowych i pliku użytkownika

```
#include <iostream>
#include <cmath>
#include "operacje.h"

int main() {
    double bok = 5;
    std::cout << "Pole kwadratu: " << poleKwadratu(bok) << std::endl;
    std::cout << "Pierwiastek z 25: " << std::sqrt(25) << std::endl;
}
```

W projekcie wieloplikowym deklaracje funkcji najczęściej umieszcza się w pliku nagłówkowym .h, a definicje funkcji w pliku .cpp. Plik main.cpp dołącza nagłówek i dzięki temu zna prototypy funkcji.

Plik nagłówkowy z zabezpieczeniem przed wielokrotnym dołączeniem

```
// operacje.h
#ifndef OPERACJE_H
#define OPERACJE_H

double poleKwadratu(double bok);
double obwodKwadratu(double bok);

#endif
```

Plik źródłowy z definicjami funkcji

```
// operacje.cpp
#include "operacje.h"

double poleKwadratu(double bok) {
    return bok * bok;
}

double obwodKwadratu(double bok) {
    return 4 * bok;
}
```

5. Dyrektywa #define

Dyrektywa #define tworzy zamiennik tekstowy. W klasycznym C i C++ używano jej do definiowania stałych, aliasów typów oraz makr przypominających funkcje. W nowoczesnym C++ część takich zastosowań ma lepsze odpowiedniki: const, constexpr, enum class, using oraz funkcje inline.

Makrostałe - historyczny sposób zapisu

```
#define PI 3.14159265
```

```
#define TRUE 1
#define FALSE 0

int wynik = (5 > 0) ? TRUE : FALSE;
double pole = PI * r * r;
```

WAŻNE: Makro nie ma typu danych. Preprocesor podstawia tekst. Kompilator zobaczy dopiero wynik podstawienia. Dlatego w C++ do stałych liczbowych częściej zaleca się constexpr double PI = 3.14159265;

Nowocześniejsze odpowiedniki w C++

```
constexpr double PI = 3.14159265;
const int MAKS_LICZBA_PROB = 3;
using Liczba = unsigned int;
```

6. Makrofunkcje i makroprocedury

Makrofunkcja to makro z parametrami, które po rozwinięciu przypomina wywołanie funkcji. Przykładem może być obliczanie kwadratu liczby. Makroprocedura to większy fragment kodu podstawiany w miejsce użycia makra, np. blok instrukcji do wczytywania danych.

Makra funkcyjne

```
#define KWADRAT(x) ((x) * (x))
#define CZY_PARZYSTA(x) (((x) % 2) == 0)

std::cout << KWADRAT(5);     // po rozwinięciu: ((5) * (5))
std::cout << CZY_PARZYSTA(8); // true
```

PUŁAPKA: Makro nie zachowuje się jak zwykła funkcja. Argument może zostać użyty więcej niż raz. Wywołanie KWADRAT(i++) może zwiększyć i dwukrotnie.

Przykład problemu z makrem

```
#define KWADRAT(x) ((x) * (x))

int i = 3;
int wynik = KWADRAT(i++); // niebezpieczne: argument i++ pojawia się dwa razy po rozwinięciu
makra
```

W programowaniu aplikacji lepszym rozwiązaniem jest zwykle funkcja inline lub szablon funkcji. Kompilator może ją zoptymalizować, ale nadal sprawdza typy i liczbę argumentów.

Bezpieczniejsza alternatywa

```
inline int kwadrat(int x) {
    return x * x;
}

template <typename T>
T kwadratSzablon(T x) {
    return x * x;
}
```

7. Dyrektywa #undef

Dyrektywa #undef usuwa wcześniejszą definicję makra. Po jej wykonaniu identyfikator nie jest już traktowany jako zamiennik preprocesora.

Po #undef blok #ifdef TEST nie zostanie włączony

```
#define TEST 1
#undef TEST

#ifdef TEST
std::cout << "Tryb testowy";
#endif
```

8. Kompilacja warunkowa

Dyrektywy kompilacji warunkowej pozwalają zdecydować, który fragment kodu trafi do kompilatora. Stosuje się je do budowania wersji debug/release, kodu zależnego od systemu operacyjnego, wariantów programu i zabezpieczania plików nagłówkowych.

Element	Znaczenie
#ifdef NAZWA	Włącza blok kodu, jeśli makro NAZWA jest zdefiniowane.
#ifndef NAZWA	Włącza blok kodu, jeśli makro NAZWA nie jest zdefiniowane.
#if warunek	Włącza blok, jeśli warunek preprocesora ma wartość różną od 0.
#elif warunek	Kolejny wariant warunku, odpowiednik else if.
#else	Blok alternatywny.
#endif	Kończy blok kompilacji warunkowej.

Warunkowe włączanie komunikatów diagnostycznych

```
#define DEBUG

#ifdef DEBUG
    std::cout << "[DEBUG] Wczytano dane" << std::endl;
#endif
```

Warunkowy kod zależny od platformy

```
#ifdef _WIN32
    std::cout << "Program kompilowany dla Windows";
#elif defined(__linux__)
    std::cout << "Program kompilowany dla Linux";
#else
    std::cout << "Inny system";
#endif
```

9. Header guard i #pragma once

Ten sam plik nagłówkowy może zostać dołączony wiele razy pośrednio przez inne pliki. Aby uniknąć wielokrotnej definicji tych samych deklaracji, stosuje się zabezpieczenie pliku nagłówkowego.

Klasyczny header guard

```
#ifndef JEDNOSTKI_H
#define JEDNOSTKI_H

double kmNaMile(double km);
double kgNaFunty(double kg);

#endif
```

Często spotykana alternatywa

```
#pragma once

double kmNaMile(double km);
double kgNaFunty(double kg);
```

EGZAMIN/PRAKTYKA: W rozwiązaniach szkolnych warto umieć odczytać obie wersje. Header guard oparty na #ifndef jest standardowym, przenośnym rozwiązaniem.

10. Dobre praktyki

- **Używaj #include świadomie.** Dołączaj tylko potrzebne pliki nagłówkowe; nadmiarowe dołączenia spowalniają kompilację i utrudniają analizę zależności.
- **Nie nadużywaj #define.** Do stałych używaj przede wszystkim const lub constexpr, a do aliasów typów using.
- **Zabezpieczaj pliki nagłówkowe.** Stosuj header guard albo #pragma once.
- **Makra nazywaj wielkimi literami.** Ułatwia to odróżnienie ich od zmiennych i funkcji.
- **Makra z parametrami zapisuj w nawiasach.** Np. #define KWADRAT(x) ((x) * (x)), aby ograniczyć błędy priorytetów operatorów.
- **Do kodu debugowania stosuj kompilację warunkową.** Pozwala zostawić w kodzie komunikaty pomocnicze, ale wyłączyć je w wersji końcowej.

11. Mini-podsumowanie

- Preprocesor działa przed kompilatorem i przetwarza dyrektywy zaczynające się od #.
- #include dołącza zawartość pliku nagłówkowego.
- #define tworzy makrodefinicję, ale w C++ często istnieją bezpieczniejsze alternatywy.
- #ifdef, #ifndef, #if, #elif, #else i #endif umożliwiają kompilację warunkową.
- Wieloplikowy projekt C++ powinien oddzielać deklaracje w plikach .h od definicji w plikach .cpp.
- Każdy nowy skrót i pojęcie należy rozumieć nie tylko z nazwy, ale też przez zastosowanie w kodzie.

12. Źródła

- Podręcznik przekazany przez nauczyciela: rozdział 9 „Preprocesor”, strony 241-254.
- cppreference.com: C++ preprocessor, conditional inclusion.
- GNU CPP Manual: The C Preprocessor, macros and conditional compilation.
- Microsoft Learn: C/C++ preprocessor directives and predefined macros.
- Materiały MEN/ZPE i informacje o kwalifikacji INF.04 dla zawodu technik programista.